Part A — Calendar Event Tracker App

This app allows you to create named events and saves them to your phone's calendar. When the event time arrives, a notification fires with Snooze (10 min) and Close buttons. Everything below is copy-paste ready.

Step 1 — Create the Project

- Open Android Studio → New Project → Empty Views Activity
- Name: CalendarEventTracker
- Package name: com.example.calendareventtracker
- Language: Java
- Minimum SDK: API 21
- Click Finish and wait for Gradle to sync

Step 2 — app/build.gradle

Open the file and make sure it looks exactly like this. Replace the entire file contents.

File: `app/build.gradle`

```
plugins {
    id 'com.android.application'
}

android {
    namespace 'com.example.calendareventtracker'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.calendareventtracker"
        minSdk 21
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
```

```
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.11.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.core:core:1.12.0'
}
```

Step 3 — AndroidManifest.xml

Replace the entire file with the content below.

File: `app/src/main/AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.READ_CALENDAR" />
    <uses-permission android:name="android.permission.WRITE_CALENDAR" />
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
    <uses-permission android:name="android.permission.USE_EXACT_ALARM" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.CalendarEventTracker">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

```

```
        <receiver
            android:name=".EventAlarmReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>

        <service
            android:name=".EventNotificationService"
            android:exported="false"
            android:foregroundServiceType="dataSync" />

    </application>
</manifest>
```

Step 4 — Resource Files

colors.xml

File: `app/src/main/res/values/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

strings.xml

File: `app/src/main/res/values/strings.xml`

```
<resources>
    <string name="app_name">CalendarEventTracker</string>
</resources>
```

themes.xml

File: `app/src/main/res/values/themes.xml`

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.CalendarEventTracker"
        parent="Theme.MaterialComponents.DayNight.DarkActionBar">
        <item name="colorPrimary">@color/purple_500</item>
```

```
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>
        <item name="android:statusBarColor"
            tools:targetApi="l">?attr/colorPrimaryVariant</item>
    </style>
</resources>
```

Step 5 — Layout File

File: `app/src/main/res/layout/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    android:background="#FAFAFA">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Calendar Event Tracker"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="#1a1a1a"
        android:paddingBottom="20dp" />

    <EditText
        android:id="@+id/etEventName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Event name"
        android:inputType="text"
        android:padding="12dp"
        android:layout_marginBottom="14dp"
        android:background="@android:drawable/edit_text" />

    <Button
        android:id="@+id/btnPickDate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Pick Date and Time"
        android:layout_marginBottom="10dp" />
```

```
    <TextView
        android:id="@+id/tvSelectedDateTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="No date / time selected yet"
        android:textSize="14sp"
        android:textColor="#555555"
        android:paddingBottom="14dp" />

    <Button
        android:id="@+id/btnAddEvent"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Add Event"
        android:layout_marginBottom="20dp" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Saved Events"
        android:textSize="16sp"
        android:textStyle="bold"
        android:paddingBottom="8dp" />

    <ListView
        android:id="@+id/lvEvents"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```

Step 6 — Java Source Files

Note: All four files go into the same package folder: app/src/main/java/com/example/calendareventtracker/

MainActivity.java

File: `app/src/main/java/com/example/calendareventtracker/MainActivity.java`

```
package com.example.calendareventtracker;

import android.Manifest;
import android.app.AlarmManager;
import android.app.DatePickerDialog;
import android.app.PendingIntent;
import android.app.TimePickerDialog;
```

```
import android.content.ContentValues;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.database.Cursor;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.provider.CalendarContract;
import android.provider.Settings;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;
import java.util.Locale;
import java.util.TimeZone;

public class MainActivity extends AppCompatActivity {

    private static final int REQ_PERMISSIONS = 101;

    private EditText etEventName;
    private TextView tvSelectedDateTime;
    private ListView lvEvents;

    private Calendar selectedCalendar = null;
    private final List<String> eventList = new ArrayList<>();
    private ArrayAdapter<String> eventAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etEventName        = findViewById(R.id.etEventName);
```

```
        tvSelectedDateTime = findViewById(R.id.tvSelectedDateTime);
        lvEvents           = findViewById(R.id.lvEvents);
        Button btnPickDate = findViewById(R.id.btnPickDate);
        Button btnAddEvent = findViewById(R.id.btnAddEvent);

        eventAdapter = new ArrayAdapter<>(this,
                android.R.layout.simple_list_item_1, eventList);
        lvEvents.setAdapter(eventAdapter);

        requestNeededPermissions();
        requestExactAlarmPermission();

        btnPickDate.setOnClickListener(v -> showDatePicker());
        btnAddEvent.setOnClickListener(v -> addEvent());
    }

    // ── Permissions ──────────────────────────────────────

    private void requestNeededPermissions() {
        List<String> needed = new ArrayList<>();
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.READ_CALENDAR)
                != PackageManager.PERMISSION_GRANTED) {
            needed.add(Manifest.permission.READ_CALENDAR);
        }
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.WRITE_CALENDAR)
                != PackageManager.PERMISSION_GRANTED) {
            needed.add(Manifest.permission.WRITE_CALENDAR);
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
                    Manifest.permission.POST_NOTIFICATIONS)
                    != PackageManager.PERMISSION_GRANTED) {
                needed.add(Manifest.permission.POST_NOTIFICATIONS);
            }
        }
        if (!needed.isEmpty()) {
            ActivityCompat.requestPermissions(this,
                    needed.toArray(new String[0]), REQ_PERMISSIONS);
        }
    }

    private void requestExactAlarmPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            AlarmManager am =
                (AlarmManager) getSystemService(Context.ALARM_SERVICE);
```

```
            if (am != null && !am.canScheduleExactAlarms()) {
                Intent i = new Intent(
                    Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
                i.setData(Uri.parse("package:" + getPackageName()));
                startActivity(i);
            }
        }
    }

    @Override
    public void onRequestPermissionsResult(int requestCode,
            @NonNull String[] permissions,
            @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(
                requestCode, permissions, grantResults);
    }

    // ── Date / Time Picker ──────────────────────────────────

    private void showDatePicker() {
        Calendar now = Calendar.getInstance();
        new DatePickerDialog(this, (view, year, month, day) ->
            new TimePickerDialog(this, (tv, hour, minute) -> {
                selectedCalendar = Calendar.getInstance();
                selectedCalendar.set(year, month, day, hour, minute, 0);
                selectedCalendar.set(Calendar.MILLISECOND, 0);
                SimpleDateFormat sdf = new SimpleDateFormat(
                    "dd MMM yyyy, hh:mm a", Locale.getDefault());
                tvSelectedDateTime.setText(
                    sdf.format(selectedCalendar.getTime()));
            }, now.get(Calendar.HOUR_OF_DAY),
               now.get(Calendar.MINUTE), false).show(),
            now.get(Calendar.YEAR),
            now.get(Calendar.MONTH),
            now.get(Calendar.DAY_OF_MONTH)).show();
    }

    // ── Add Event ───────────────────────────────────────────

    private void addEvent() {
        String name = etEventName.getText().toString().trim();
        if (name.isEmpty()) {
            Toast.makeText(this,
                "Enter an event name", Toast.LENGTH_SHORT).show();
            return;
        }
        if (selectedCalendar == null) {
```

```
            Toast.makeText(this,
                "Pick a date and time first", Toast.LENGTH_SHORT).show();
            return;
        }
        if (selectedCalendar.getTimeInMillis()
                <= System.currentTimeMillis()) {
            Toast.makeText(this,
                "Choose a future date/time", Toast.LENGTH_SHORT).show();
            return;
        }

        long calId = getDefaultCalendarId();
        if (calId == -1) {
            Toast.makeText(this,
                "No calendar found. Sign into a Google account first.",
                Toast.LENGTH_LONG).show();
            return;
        }

        long eventId = insertCalendarEvent(
            calId, name, selectedCalendar.getTimeInMillis());
        if (eventId == -1) {
            Toast.makeText(this,
                "Could not save event. Check calendar permission.",
                Toast.LENGTH_LONG).show();
            return;
        }

        scheduleAlarm(eventId, name, selectedCalendar.getTimeInMillis());

        SimpleDateFormat sdf = new SimpleDateFormat(
            "dd MMM yyyy, hh:mm a", Locale.getDefault());
        eventList.add(name + "  —  "
            + sdf.format(selectedCalendar.getTime()));
        eventAdapter.notifyDataSetChanged();

        etEventName.setText("");
        tvSelectedDateTime.setText("No date / time selected yet");
        selectedCalendar = null;
        Toast.makeText(this, "Event saved!", Toast.LENGTH_SHORT).show();
    }

    // ── Calendar Helpers ──────────────────────────────────

    private long getDefaultCalendarId() {
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.READ_CALENDAR)
```

```
                != PackageManager.PERMISSION_GRANTED) return -1;

        String[] proj = { CalendarContract.Calendars._ID };
        // Try primary calendar first
        try (Cursor c = getContentResolver().query(
                CalendarContract.Calendars.CONTENT_URI, proj,
                CalendarContract.Calendars.IS_PRIMARY + "=1",
                null, null)) {
            if (c != null && c.moveToFirst()) return c.getLong(0);
        } catch (Exception ignored) { }
        // Fall back to any calendar
        try (Cursor c = getContentResolver().query(
                CalendarContract.Calendars.CONTENT_URI,
                proj, null, null, null)) {
            if (c != null && c.moveToFirst()) return c.getLong(0);
        } catch (Exception ignored) { }
        return -1;
    }

    private long insertCalendarEvent(
            long calendarId, String title, long startMs) {
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.WRITE_CALENDAR)
                != PackageManager.PERMISSION_GRANTED) return -1;

        ContentValues cv = new ContentValues();
        cv.put(CalendarContract.Events.CALENDAR_ID, calendarId);
        cv.put(CalendarContract.Events.TITLE, title);
        cv.put(CalendarContract.Events.DTSTART, startMs);
        cv.put(CalendarContract.Events.DTEND,
               startMs + 60 * 60 * 1000L); // 1-hour duration
        cv.put(CalendarContract.Events.EVENT_TIMEZONE,
               TimeZone.getDefault().getID());
        try {
            Uri uri = getContentResolver().insert(
                CalendarContract.Events.CONTENT_URI, cv);
            if (uri != null)
                return Long.parseLong(uri.getLastPathSegment());
        } catch (Exception e) { e.printStackTrace(); }
        return -1;
    }

    // ── Alarm Scheduling ──────────────────────────────────

    private void scheduleAlarm(
            long eventId, String title, long triggerMs) {
        AlarmManager am =
```

```
            (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        if (am == null) return;

        Intent intent = new Intent(this, EventAlarmReceiver.class);
        intent.putExtra("event_id", eventId);
        intent.putExtra("event_title", title);
        intent.putExtra("trigger_millis", triggerMs);

        PendingIntent pi = PendingIntent.getBroadcast(
            this, (int) eventId, intent,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            am.setExactAndAllowWhileIdle(
                AlarmManager.RTC_WAKEUP, triggerMs, pi);
        } else {
            am.setExact(AlarmManager.RTC_WAKEUP, triggerMs, pi);
        }
    }
}
```

EventAlarmReceiver.java

File: `app/src/main/java/com/example/calendareventtracker/EventAlarmReceiver.java`

```
package com.example.calendareventtracker;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Build;

public class EventAlarmReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {
        // Ignore boot broadcasts (no persistent DB in this demo)
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction()))
            return;

        long   eventId  = intent.getLongExtra("event_id", -1);
        String title    = intent.getStringExtra("event_title");
        long   trigMs   = intent.getLongExtra("trigger_millis", 0);
        if (title == null) title = "Event Reminder";

        Intent svcIntent =
            new Intent(context, EventNotificationService.class);
```

```
        svcIntent.putExtra("event_id",       eventId);
        svcIntent.putExtra("event_title",    title);
        svcIntent.putExtra("trigger_millis", trigMs);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            context.startForegroundService(svcIntent);
        } else {
            context.startService(svcIntent);
        }
    }
}
```

EventNotificationService.java

File: `app/src/main/java/com/example/calendareventtracker/EventNotificationService.java`

```
package com.example.calendareventtracker;

import android.app.AlarmManager;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.Service;
import android.content.Context;
import android.content.Intent;
import android.os.Build;
import android.os.IBinder;

import androidx.core.app.NotificationCompat;

public class EventNotificationService extends Service {

    public static final String CHANNEL_ID   =
        "calendar_event_channel";
    public static final String ACTION_SNOOZE =
        "com.example.calendareventtracker.SNOOZE";
    public static final String ACTION_CLOSE  =
        "com.example.calendareventtracker.CLOSE";
    private static final int   NOTIF_ID     = 2001;

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        createChannel();

        if (intent == null) { stopSelf(); return START_NOT_STICKY; }

        String action = intent.getAction();
```

```
        if (ACTION_SNOOZE.equals(action)) {
            snooze(intent);
            stopSelf();
            return START_NOT_STICKY;
        }
        if (ACTION_CLOSE.equals(action)) {
            stopSelf();
            return START_NOT_STICKY;
        }

        // Normal fire — show notification
        long   eventId = intent.getLongExtra("event_id", -1);
        String title   = intent.getStringExtra("event_title");
        long   trigMs  = intent.getLongExtra("trigger_millis", 0);
        if (title == null) title = "Event Reminder";

        PendingIntent snoozePi = buildServicePi(
            ACTION_SNOOZE, eventId, title, trigMs, 1);
        PendingIntent closePi  = buildServicePi(
            ACTION_CLOSE,  eventId, title, trigMs, 2);

        Notification notif = new NotificationCompat
                .Builder(this, CHANNEL_ID)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle("Event Reminder")
            .setContentText(title)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setCategory(NotificationCompat.CATEGORY_ALARM)
            .setOngoing(true)
            .setAutoCancel(false)
            .addAction(android.R.drawable.ic_popup_reminder,
                       "Snooze 10 min", snoozePi)
            .addAction(android.R.drawable.ic_menu_close_clear_cancel,
                       "Close", closePi)
            .build();

        startForeground(NOTIF_ID, notif);
        return START_NOT_STICKY;
    }

    private PendingIntent buildServicePi(String action,
            long eventId, String title, long trigMs, int offset) {
        Intent i = new Intent(this, EventNotificationService.class);
        i.setAction(action);
        i.putExtra("event_id",       eventId);
        i.putExtra("event_title",    title);
```

```
        i.putExtra("trigger_millis", trigMs);
        return PendingIntent.getService(
            this, (int)(eventId + offset * 1000), i,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);
    }

    private void snooze(Intent intent) {
        long   eventId = intent.getLongExtra("event_id", -1);
        String title   = intent.getStringExtra("event_title");
        long   trigMs  = intent.getLongExtra("trigger_millis", 0);
        if (title == null) title = "Event Reminder";

        long snoozeMs = System.currentTimeMillis() + 10 * 60 * 1000L;

        AlarmManager am =
            (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        if (am == null) return;

        Intent recv = new Intent(this, EventAlarmReceiver.class);
        recv.putExtra("event_id",       eventId);
        recv.putExtra("event_title",    title);
        recv.putExtra("trigger_millis", snoozeMs);

        PendingIntent pi = PendingIntent.getBroadcast(
            this, (int) eventId, recv,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            am.setExactAndAllowWhileIdle(
                AlarmManager.RTC_WAKEUP, snoozeMs, pi);
        } else {
            am.setExact(AlarmManager.RTC_WAKEUP, snoozeMs, pi);
        }
    }

    private void createChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel ch = new NotificationChannel(
                CHANNEL_ID,
                "Calendar Event Reminders",
                NotificationManager.IMPORTANCE_HIGH);
            ch.setDescription("Reminders for calendar events");
            ch.enableVibration(true);
            NotificationManager nm =
                getSystemService(NotificationManager.class);
```

```
            if (nm != null) nm.createNotificationChannel(ch);
        }
    }

    @Override
    public IBinder onBind(Intent intent) { return null; }
}
```

Step 7 — Running Part A

- Use a real device or an emulator that has a Google account signed in (required for the Calendar provider).
- Run the app. Tap Allow on every permission dialog that appears.
- On Android 12 or higher, the app opens the system Exact Alarm settings — tap Allow for this app.
- Type an event name, tap Pick Date and Time, select a time a couple of minutes from now, then tap Add Event.
- Lock your screen and wait. When the time arrives, a persistent notification appears with Snooze 10 min and Close buttons.
- Snooze reschedules the same notification 10 minutes later. Close dismisses it permanently.

Part B — Alarm Clock Reminder App

This is a standalone alarm clock. You give the alarm a label and set a time. When the alarm fires, a persistent notification with Snooze and Close actions appears. No calendar account is needed.

Step 1 — Create the Project

- New Project → Empty Views Activity
- Name: AlarmClockReminder
- Package name: com.example.alarmclockreminder
- Language: Java
- Minimum SDK: API 21
- Click Finish

Step 2 — app/build.gradle

File: `app/build.gradle`

```
plugins {
    id 'com.android.application'
}

android {
    namespace 'com.example.alarmclockreminder'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.alarmclockreminder"
        minSdk 21
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile(
                'proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
```

```
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.11.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.core:core:1.12.0'
}
```

Step 3 — AndroidManifest.xml

File: `app/src/main/AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
    <uses-permission android:name="android.permission.USE_EXACT_ALARM" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AlarmClockReminder">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <receiver
            android:name=".AlarmReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>

        <service
```

```
            android:name=".AlarmNotificationService"
            android:exported="false"
            android:foregroundServiceType="dataSync" />

    </application>
</manifest>
```

Step 4 — Resource Files

colors.xml

File: `app/src/main/res/values/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

strings.xml

File: `app/src/main/res/values/strings.xml`

```
<resources>
    <string name="app_name">AlarmClockReminder</string>
</resources>
```

themes.xml

File: `app/src/main/res/values/themes.xml`

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.AlarmClockReminder"
        parent="Theme.MaterialComponents.DayNight.DarkActionBar">
        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>
        <item name="android:statusBarColor"
            tools:targetApi="l">?attr/colorPrimaryVariant</item>
    </style>
</resources>
```

Step 5 — Layout File

File: `app/src/main/res/layout/activity_main.xml`

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    android:background="#FAFAFA">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Alarm Clock Reminder"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="#1a1a1a"
        android:paddingBottom="20dp" />

    <EditText
        android:id="@+id/etAlarmLabel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Alarm label (e.g. Meeting, Medicine)"
        android:inputType="text"
        android:padding="12dp"
        android:layout_marginBottom="14dp"
        android:background="@android:drawable/edit_text" />

    <Button
        android:id="@+id/btnPickTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Pick Date and Time"
        android:layout_marginBottom="10dp" />

    <TextView
        android:id="@+id/tvSelectedTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="No time selected yet"
        android:textSize="14sp"
        android:textColor="#555555"
        android:paddingBottom="14dp" />
```

```

    <Button
        android:id="@+id/btnSetAlarm"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Set Alarm"
        android:layout_marginBottom="20dp" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Scheduled Alarms"
        android:textSize="16sp"
        android:textStyle="bold"
        android:paddingBottom="8dp" />

    <ListView
        android:id="@+id/lvAlarms"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```

Step 6 — Java Source Files

Note: All three files go into: app/src/main/java/com/example/alarmclockreminder/

MainActivity.java

File: `app/src/main/java/com/example/alarmclockreminder/MainActivity.java`

```
package com.example.alarmclockreminder;

import android.Manifest;
import android.app.AlarmManager;
import android.app.DatePickerDialog;
import android.app.PendingIntent;
import android.app.TimePickerDialog;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.provider.Settings;
import android.widget.ArrayAdapter;
import android.widget.Button;
```

```
import android.widget.EditText;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.atomic.AtomicInteger;

public class MainActivity extends AppCompatActivity {

    private static final int REQ_NOTIF = 201;
    // Simple counter for unique PendingIntent request codes
    private static final AtomicInteger alarmCounter =
        new AtomicInteger(0);

    private EditText etAlarmLabel;
    private TextView tvSelectedTime;
    private ListView lvAlarms;

    private Calendar selectedCalendar = null;
    private final List<String> alarmList = new ArrayList<>();
    private ArrayAdapter<String> alarmAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etAlarmLabel   = findViewById(R.id.etAlarmLabel);
        tvSelectedTime = findViewById(R.id.tvSelectedTime);
        lvAlarms       = findViewById(R.id.lvAlarms);
        Button btnPickTime = findViewById(R.id.btnPickTime);
        Button btnSetAlarm = findViewById(R.id.btnSetAlarm);

        alarmAdapter = new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1, alarmList);
        lvAlarms.setAdapter(alarmAdapter);
```

```
        requestNotificationPermission();
        requestExactAlarmPermission();

        btnPickTime.setOnClickListener(v -> showDatePicker());
        btnSetAlarm.setOnClickListener(v -> setAlarm());
    }

    // —— Permissions ————————————————————————————————————

    private void requestNotificationPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
                    Manifest.permission.POST_NOTIFICATIONS)
                    != PackageManager.PERMISSION_GRANTED) {
                ActivityCompat.requestPermissions(this,
                    new String[]{Manifest.permission.POST_NOTIFICATIONS},
                    REQ_NOTIF);
            }
        }
    }

    private void requestExactAlarmPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            AlarmManager am =
                (AlarmManager) getSystemService(Context.ALARM_SERVICE);
            if (am != null && !am.canScheduleExactAlarms()) {
                Intent i = new Intent(
                    Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
                i.setData(Uri.parse("package:" + getPackageName()));
                startActivity(i);
            }
        }
    }

    @Override
    public void onRequestPermissionsResult(int requestCode,
            @NonNull String[] permissions,
            @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(
            requestCode, permissions, grantResults);
    }

    // —— Date / Time Picker ——————————————————————————————

    private void showDatePicker() {
        Calendar now = Calendar.getInstance();
        new DatePickerDialog(this, (v, year, month, day) ->
```

```
            new TimePickerDialog(this, (tv, hour, minute) -> {
                selectedCalendar = Calendar.getInstance();
                selectedCalendar.set(year, month, day, hour, minute, 0);
                selectedCalendar.set(Calendar.MILLISECOND, 0);
                SimpleDateFormat sdf = new SimpleDateFormat(
                    "dd MMM yyyy, hh:mm a", Locale.getDefault());
                tvSelectedTime.setText(
                    sdf.format(selectedCalendar.getTime()));
            }, now.get(Calendar.HOUR_OF_DAY),
               now.get(Calendar.MINUTE), false).show(),
            now.get(Calendar.YEAR),
            now.get(Calendar.MONTH),
            now.get(Calendar.DAY_OF_MONTH)).show();
    }

    // ── Set Alarm ──────────────────────────────────────────

    private void setAlarm() {
        String label = etAlarmLabel.getText().toString().trim();
        if (label.isEmpty()) {
            Toast.makeText(this,
                "Enter an alarm label", Toast.LENGTH_SHORT).show();
            return;
        }
        if (selectedCalendar == null) {
            Toast.makeText(this,
                "Pick a date and time first", Toast.LENGTH_SHORT).show();
            return;
        }
        if (selectedCalendar.getTimeInMillis()
                <= System.currentTimeMillis()) {
            Toast.makeText(this,
                "Choose a future date/time", Toast.LENGTH_SHORT).show();
            return;
        }

        int reqCode = alarmCounter.incrementAndGet();
        scheduleAlarm(reqCode, label,
            selectedCalendar.getTimeInMillis());

        SimpleDateFormat sdf = new SimpleDateFormat(
            "dd MMM yyyy, hh:mm a", Locale.getDefault());
        alarmList.add(label + "  —  "
            + sdf.format(selectedCalendar.getTime()));
        alarmAdapter.notifyDataSetChanged();

        etAlarmLabel.setText("");
```

```
        tvSelectedTime.setText("No time selected yet");
        selectedCalendar = null;
        Toast.makeText(this, "Alarm set!", Toast.LENGTH_SHORT).show();
    }

    // ── Alarm Scheduling ──────────────────────────────────

    static void scheduleAlarm(int requestCode,
            String label, long triggerMs) {
        // Static so AlarmReceiver can call it when rescheduling snooze
        // But we need a Context — see AlarmReceiver.java for snooze logic
    }

    private void scheduleAlarm(
            int requestCode, String label, long triggerMs) {
        AlarmManager am =
            (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        if (am == null) return;

        Intent intent = new Intent(this, AlarmReceiver.class);
        intent.putExtra("request_code", requestCode);
        intent.putExtra("alarm_label",  label);
        intent.putExtra("trigger_millis", triggerMs);

        PendingIntent pi = PendingIntent.getBroadcast(
            this, requestCode, intent,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            am.setExactAndAllowWhileIdle(
                AlarmManager.RTC_WAKEUP, triggerMs, pi);
        } else {
            am.setExact(AlarmManager.RTC_WAKEUP, triggerMs, pi);
        }
    }
}
```

AlarmReceiver.java

File: `app/src/main/java/com/example/alarmclockreminder/AlarmReceiver.java`

```
package com.example.alarmclockreminder;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Build;
```

```java

public class AlarmReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction()))
            return; // No persistent DB in this demo

        int    reqCode  = intent.getIntExtra("request_code", 0);
        String label    = intent.getStringExtra("alarm_label");
        long   trigMs   = intent.getLongExtra("trigger_millis", 0);
        if (label == null) label = "Alarm";

        Intent svc =
            new Intent(context, AlarmNotificationService.class);
        svc.putExtra("request_code",   reqCode);
        svc.putExtra("alarm_label",    label);
        svc.putExtra("trigger_millis", trigMs);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            context.startForegroundService(svc);
        } else {
            context.startService(svc);
        }
    }
}
```

AlarmNotificationService.java

File: `app/src/main/java/com/example/alarmclockreminder/AlarmNotificationService.java`

```java
package com.example.alarmclockreminder;

import android.app.AlarmManager;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.Service;
import android.content.Context;
import android.content.Intent;
import android.os.Build;
import android.os.IBinder;

import androidx.core.app.NotificationCompat;

public class AlarmNotificationService extends Service {

```

```
    public static final String CHANNEL_ID    = "alarm_channel";
    public static final String ACTION_SNOOZE =
        "com.example.alarmclockreminder.SNOOZE";
    public static final String ACTION_CLOSE  =
        "com.example.alarmclockreminder.CLOSE";
    private static final int   NOTIF_ID      = 3001;

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        createChannel();

        if (intent == null) { stopSelf(); return START_NOT_STICKY; }

        String action = intent.getAction();

        if (ACTION_SNOOZE.equals(action)) {
            snooze(intent);
            stopSelf();
            return START_NOT_STICKY;
        }
        if (ACTION_CLOSE.equals(action)) {
            stopSelf();
            return START_NOT_STICKY;
        }

        // Normal alarm fire
        int    reqCode = intent.getIntExtra("request_code", 0);
        String label   = intent.getStringExtra("alarm_label");
        long   trigMs  = intent.getLongExtra("trigger_millis", 0);
        if (label == null) label = "Alarm";

        PendingIntent snoozePi =
            buildSvcPi(ACTION_SNOOZE, reqCode, label, trigMs, 10000);
        PendingIntent closePi  =
            buildSvcPi(ACTION_CLOSE,  reqCode, label, trigMs, 20000);

        Notification notif = new NotificationCompat
                .Builder(this, CHANNEL_ID)
            .setSmallIcon(android.R.drawable.ic_lock_idle_alarm)
            .setContentTitle("Alarm: " + label)
            .setContentText("Time is up!")
            .setPriority(NotificationCompat.PRIORITY_MAX)
            .setCategory(NotificationCompat.CATEGORY_ALARM)
            .setOngoing(true)
            .setAutoCancel(false)
            .addAction(android.R.drawable.ic_popup_reminder,
                       "Snooze 10 min", snoozePi)
```

```
            .addAction(android.R.drawable.ic_menu_close_clear_cancel,
                       "Close", closePi)
            .build();

        startForeground(NOTIF_ID, notif);
        return START_NOT_STICKY;
    }

    private PendingIntent buildSvcPi(String action,
            int reqCode, String label, long trigMs, int offset) {
        Intent i = new Intent(this, AlarmNotificationService.class);
        i.setAction(action);
        i.putExtra("request_code",   reqCode);
        i.putExtra("alarm_label",    label);
        i.putExtra("trigger_millis", trigMs);
        return PendingIntent.getService(
            this, reqCode + offset, i,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);
    }

    private void snooze(Intent intent) {
        int    reqCode = intent.getIntExtra("request_code", 0);
        String label   = intent.getStringExtra("alarm_label");
        long   trigMs  = intent.getLongExtra("trigger_millis", 0);
        if (label == null) label = "Alarm";

        long snoozeMs = System.currentTimeMillis() + 10 * 60 * 1000L;

        AlarmManager am =
            (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        if (am == null) return;

        Intent recv = new Intent(this, AlarmReceiver.class);
        recv.putExtra("request_code",   reqCode);
        recv.putExtra("alarm_label",    label);
        recv.putExtra("trigger_millis", snoozeMs);

        PendingIntent pi = PendingIntent.getBroadcast(
            this, reqCode, recv,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            am.setExactAndAllowWhileIdle(
                AlarmManager.RTC_WAKEUP, snoozeMs, pi);
        } else {
```

```
            am.setExact(AlarmManager.RTC_WAKEUP, snoozeMs, pi);
        }
    }

    private void createChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel ch = new NotificationChannel(
                CHANNEL_ID, "Alarm Reminders",
                NotificationManager.IMPORTANCE_HIGH);
            ch.setDescription("Alarm clock notifications");
            ch.enableVibration(true);
            NotificationManager nm =
                getSystemService(NotificationManager.class);
            if (nm != null) nm.createNotificationChannel(ch);
        }
    }

    @Override
    public IBinder onBind(Intent intent) { return null; }
}
```

Step 7 — Running Part B

- Run the app on a device or emulator (no Google account needed for this one).
- Grant the Notification permission when prompted.
- On Android 12+, tap Allow in the exact alarm settings screen that opens automatically.
- Type a label (e.g. Take Medicine), pick a time a couple of minutes from now, tap Set Alarm.
- Lock the screen and wait. The notification fires with Snooze 10 min and Close buttons.
- Snooze reschedules the alarm 10 minutes later. Close dismisses it.

Quick Troubleshooting Reference

If something does not work, check these common issues before anything else.

Build errors

- If Gradle says it cannot resolve a dependency, go to File → Sync Project with Gradle Files and try again. Make sure you have an internet connection.
- If you see 'Unresolved reference' errors, confirm the package name at the top of each Java file exactly matches the folder path and what you entered during project creation.
- If the theme is missing, make sure the style name in themes.xml matches what is in AndroidManifest.xml — they must be identical.

App crashes on launch

- This almost always means a missing permission in the manifest or a wrong package name somewhere. Double-check that all four files in Part A (or three in Part B) share the same package name.
- Check Logcat in Android Studio for the exact line that caused the crash. Filter by your package name.

Notification does not appear

- On Android 13 (API 33) and higher, the POST_NOTIFICATIONS permission is required at runtime. Make sure you tapped Allow when the dialog appeared.
- On Android 12 (API 31) and higher, exact alarms require special permission. If the settings screen appeared and you did not allow it, go to Settings → Apps → your app → Alarms and Reminders and enable it.
- Check that your device is not in Do Not Disturb mode, which blocks notifications.

Calendar event not saved (Part A only)

- A Google or other calendar account must be signed in on the device. Open the built-in Calendar app — if it shows nothing, sign in first.
- Both READ_CALENDAR and WRITE_CALENDAR must be granted. Go to Settings → Apps → CalendarEventTracker → Permissions and confirm both are on.

Alarm fires immediately or not at all

- Make sure you selected a time in the future. The app shows a toast if you pick a past time, but double-check the AM/PM selection in the time picker.
- On some manufacturer-specific Android builds (especially Xiaomi, Huawei, Samsung), battery optimisation kills background services. Go to Settings → Battery → App battery usage → your app → set to Unrestricted.